

TECHNICAL SOLUTION GUIDE ·
PACKAGE REVAMP 1.4.0

QZ Tray Zero-Prompt Trust Architecture

Eliminate every browser certificate warning and "Allow" dialog for multi-tenant Laravel printing — with self-signed certificates, your own free root CA, and silent Windows deployment.

[bitdreamit/laravel-qz-tray](#) · v1.4.0

Windows clients · Chrome/Edge · wildcard
subdomain tenancy

BIT DREAM IT · SEPTEMBER 2026

Table of Contents

1	Executive Summary	1
2	Root-Cause Analysis: Why "Allow" Appears Every Time	2
3	How QZ Tray Trust Actually Works	4
4	The Free Zero-Prompt Architectures	6
5	Recommended Blueprint: Own CA + Wildcard + Client Bundle	7
6	Package Revamp v1.4.0	9
7	Windows Client Deployment	11
8	Renewal, Rotation and Monitoring	13
9	Troubleshooting Matrix and Verification Checklist	13
	Appendix A QZ Tray Quick Reference	14
	Appendix B New Environment Variables	15

1 Executive Summary

This guide answers one question: how can a Laravel SaaS that serves many tenant subdomains print through QZ Tray without a single certificate prompt, without paying for a signing certificate, and without asking shop staff to click anything more than once per machine. The answer is documented here as a full analysis of the **bitdreamit/laravel-qz-tray** package together with a complete, backward-compatible revamp (v1.4.0) that automates every trust decision QZ Tray can make.

The recurring "Allow" dialog is not a bug in your code. It is the correct behaviour of QZ Tray's two-layer security model when it is fed a self-signed certificate that chains to no trusted root, combined with end users who click the session-only *Allow* button instead of *Always Allow*. Because every tenant subdomain previously generated its own certificate, each new subdomain presented a new fingerprint, and every client machine had to re-approve it. The model below removes that entire class of problems.

3 prompts

TLS warning, signing dialog,
local-network permission

3 fixes

Windows Root store, override.crt,
browser policy

\$0 cost

own CA or Let's Encrypt - no paid QZ
certificate

Three free paths achieve zero prompts. Path A imports a real CA certificate (Let's Encrypt or cPanel AutoSSL), which QZ Tray trusts silently because it chains to a public root. Path B, the recommended blueprint for your environment, creates your own root CA with a new artisan command and deploys it once per Windows client as QZ Tray's *override.crt* through a generated PowerShell bundle. Path C keeps the shared self-signed pair and relies on one correctly clicked "Always Allow" per machine. All three are implemented, documented and testable in v1.4.0.

Decision summary

Decision	Choose this when	Client-side work	Renewal story
Path A - real CA	Servers run certbot or cPanel AutoSSL already	None for signing	Silent; auto re-import via qz:watch-certificate
Path B - own CA	You control Windows clients (GPO/Intune available)	One bundle run per machine	Leaf rotation invisible to clients
Path C - shared self-signed	No admin rights, minimal infrastructure	One "Always Allow" per machine	Regeneration re-prompts every client

Table 1-1. Which zero-prompt path fits which deployment. Paths can be combined.

Everything in this document was validated against the official QZ Tray 2.2.x documentation (provisioning, command-line, print-server and signing references). The v1.4.0 code ships with openssl-level tests that prove the generated chain verifies, and the new Client Setup Wizard gives your support team a single URL - **/qz/setup** - that reports the trust posture of any machine and walks an operator to a prompt-free state.

2 Root-Cause Analysis: Why "Allow" Appears Every Time

Users of the package reported that printing "needs allow it every time, very bad". Reproducing that behaviour requires looking at three separate trust decisions that happen on the way from a tenant web page to a physical printer. None of them is a domain problem, and only one of them is under your server's direct control.

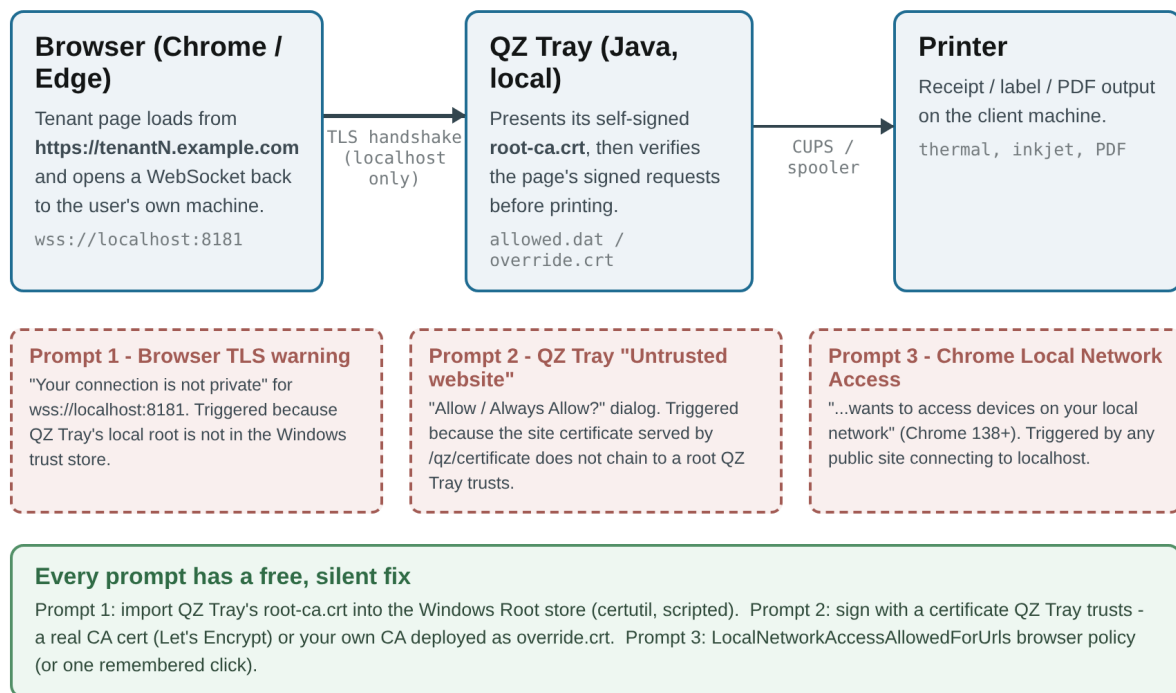


Figure 2-1. The three independent trust decisions between a tenant page and paper.

2.1 The fingerprint, not the domain

QZ Tray's signing prompt is keyed to the SHA-1 fingerprint of the certificate a site presents, never to the host name. This single fact explains most of the historical pain: when each tenant vhost ran **qz:generate-certificate** independently, every subdomain produced a different self-signed certificate, so each tenant was a stranger to QZ Tray even on the same machine. The v1.3.0 release already fixed the server half by letting all vhosts share one pair through **QZ_CERT_PATH/QZ_KEY_PATH** or the export/import commands. The client half - making QZ Tray trust that one fingerprint silently - is what v1.4.0 completes.

2.2 Allow versus Always Allow

The QZ Tray dialog offers two buttons with very different lifetimes. *Allow* approves the request for the current session only; close the tab or restart the tray and the same question returns. *Always Allow* ("Remember this decision") writes the certificate into **allowed.dat** in the user's QZ data directory, after which the prompt never appears again for that fingerprint. Non-technical operators reasonably click the left button. No server-side configuration can change which button a user presses - but the architectures in chapter 4 remove the dialog from the flow entirely, which is the only reliable fix.

2.3 Package-specific findings

The code audit confirmed the signing implementation itself is correct and idiomatic: `/qz/sign` signs the SHA-512 digest server-side with openssl, `/qz/certificate` serves the PEM, and smart-print.js wires both promises properly. The gaps were all on the trust lifecycle side.

Audit findings

#	Finding	Consequence before v1.4.0	Fix shipped in v1.4.0
F1	Per-vhost certificates before v1.3.0	A new prompt per tenant subdomain	Shared paths + wildcard leaf with SANs
F2	Self-signed leaf by default	"Untrusted website" dialog on every new machine	Own-CA chain + override.crt deployment
F3	No client trust tooling	Trust existed only as user education in docs	qz:client-bundle PowerShell deployment
F4	No renewal automation	Let's Encrypt renewals silently drifted out of <code>/qz/certificate</code>	qz:watch-certificate with daily scheduling
F5	No browser-facing diagnostics	Support could not see what the client sees	GET <code>/qz/setup</code> wizard with live probe
F6	Users clicked session-only Allow	Prompts "every time" even with a shared cert	Zero-dialog paths + wizard guidance

Table 2-1. Trust-lifecycle gaps found in the package and their resolution.

3 How QZ Tray Trust Actually Works

QZ Tray listens on the user's own machine - never on your server. A page at <https://tenantN.example.com> opens a secure WebSocket to **wss://localhost:8181** and talks to the locally installed Java application. Understanding which component evaluates which certificate makes every prompt predictable.

3.1 Layer one - transport TLS on localhost

At install time QZ Tray generates a self-signed certificate for localhost traffic, stored as **root-ca.crt** in the shared data directory (C:\ProgramData\qz on Windows) together with a Java keystore. Chrome and Edge validate that certificate against the Windows certificate store. When the store does not contain it, the browser shows the red "not private" interstitial or, on newer builds, silently refuses the WebSocket - and users blame your application. Installing QZ Tray's own root into the Local Machine Root store with `certutil` fixes this permanently, and QZ Tray's installer attempts it automatically when elevated rights are available. The v1.4.0 bundle re-asserts the step so it is never the reason a cashier sees a warning.

3.2 Layer two - request signing

Every sensitive call (print, find printers, serial, USB) must be signed. The browser fetches your public certificate from **/qz/certificate** through `setCertificatePromise`, and obtains a base64 SHA-512 signature per request from **/qz/sign** through `setSignaturePromise`. QZ Tray verifies the signature against the certificate and then decides whether to trust the certificate itself. A certificate is trusted without any dialog when it chains to one of: a public root already in the Java trust store (any Let's Encrypt or commercial CA certificate), the root configured as **override.crt** in the QZ Tray install directory or via **authcert.override** in `qz-tray.properties`, or an entry in the per-user **allowed.dat** whitelist. Everything else raises the "Untrusted website" dialog.

3.3 Layer three - Chrome Local Network Access

Since Chrome 138 (2025), a public HTTPS site that connects to localhost triggers a one-time "wants to access your local network" permission. It is rememberable, so users see it once per site, but in a tenanted fleet that is once per tenant domain. The browser policy **LocalNetworkAccessAllowedForUrls** pre-approves listed domains, and QZ Tray 2.2.6's provisioning format can install the Chromium and Firefox variants of that policy - both are included in the generated bundle.

The three trust decisions

Prompt	Decided by	Trusts when	Silent fix
TLS warning (wss://localhost:8181)	Browser + Windows store	QZ Tray's root-ca.crt is in the Root store	certutil -addstore Root (scripted)
"Untrusted website" dialog	QZ Tray (Java)	Cert chains to public root, override.crt, or allowed.dat	own CA as override.crt / real CA cert
Local network permission	Chrome/Edge policy	Site listed in LocalNetworkAccessAllowedForUrls	Registry policy via bundle

Table 3-1. Who decides, and how each decision is pre-approved.

3.4 Files and ports on a Windows client

All client-side automation in this guide touches only these paths. Knowing them also makes support tickets specific: an operator can screenshot the exact folder.

Windows file map

File	Path (Windows)	Purpose
root-ca.crt	C:\ProgramData\qz\root-ca.crt	QZ Tray's TLS root for wss://localhost:8181
override.crt	C:\Program Files\QZ Tray\override.crt	Replaces the signing trust root with yours
qz-tray.properties	C:\Program Files\QZ Tray\qz-tray.properties	authcert.override=... property form
allowed.dat	C:\Users\%AppData%\Roaming\qz\allowed.dat	"Always Allow" fingerprints per user
provision/	C:\Program Files\QZ Tray\provision\	Sideload folder for provision.json (2.2.4+)
qz-tray-console.exe	C:\Program Files\QZ Tray\	CLI: --allow, certgen, console logging

Table 3-2. Client-side files referenced by the deployment scripts.

4 The Free Zero-Prompt Architectures

All five mechanisms below are free. They differ in where trust is created - on the server, on the client, or by one human click - and that difference determines how much of the fleet you must touch when certificates rotate.

4.1 Path A - a real CA certificate (zero client deployment)

Import the same fullchain certificate your web server already uses for HTTPS as the QZ signing certificate. QZ Tray 2.2 bundles OpenJDK 11 whose trust store contains ISRG Root X1, so a Let's Encrypt chain is verified silently - no `override.crt`, no scripts, no client contact at all. The only operational requirement is freshness: when `certbot` renews the leaf, `/qz/certificate` must not keep serving the old one. The new **qz:watch-certificate** command closes that loop by detecting the change and re-importing atomically, and the service provider schedules it daily as soon as a watch source is configured.

```
Path A - server side only
php artisan qz:certificate:import
--cert=/etc/letsencrypt/live/app.example.com/fullchain.pem \
--key=/etc/letsencrypt/live/app.example.com/privkey.pem
# .env
QZ_WATCH_CERT=/etc/letsencrypt/live/app.example.com/fullchain.pem
QZ_WATCH_KEY=/etc/letsencrypt/live/app.example.com/privkey.pem
# scheduled automatically once QZ_WATCH_CERT is set:
php artisan qz:watch-certificate # re-imports after every renewal
```

4.2 Path B - your own root CA + `override.crt` (recommended here)

Create a dedicated root CA that belongs to your project, sign a wildcard leaf for `*.example.com` with it, and distribute the CA certificate to clients as QZ Tray's `override.crt`. QZ Tray then verifies your leaf the same way a browser verifies a public certificate - by chaining to a root it has been told to trust - and the dialog never appears. This is functionally the free equivalent of the paid signing certificate QZ Industries sells. Its key advantage over Path A is rotation isolation: leaves can be reissued at any time without touching a single client, because trust is pinned to the long-lived root. With full admin control of your Windows fleet (GPO, Intune or a logon script), the one-time deployment is invisible.

4.3 Path C - shared self-signed + Always Allow

The lightest path: keep the self-signed pair, make certain every tenant serves the identical files (shared `QZ_CERT_PATH/QZ_KEY_PATH`, or `qz:certificate:export` to a `.pfx` and import on each vhost), and have each machine click "Always Allow" exactly once. Because the fingerprint is identical across tenants, that single click covers all of them. This is also the correct fallback for BYOD machines where no admin deployment is possible. The zero-deploy variant exports the leaf itself as `override.crt` - workable, but prefer Path B so a future leaf rotation does not require re-touching clients.

4.4 Comparison and rotation rules

The decisive question is not "can the prompt be removed" - all three paths remove it - but "what happens when a certificate changes". The table below summarises the fleet impact of each routine operation.

Fleet impact comparison

Operation	Path A (real CA)	Path B (own CA)	Path C (self-signed)
Initial client setup	Nothing	One bundle run (silent)	One "Always Allow" click
Leaf renewal / reissue	Silent; watcher re-imports	Silent; chain unchanged	One re-approval per machine
Adding a tenant subdomain	Nothing (same pair or per-host import)	Nothing (wildcard covers it)	Nothing if pair shared
CA rotation	Not applicable	Redeploy override.crt first	Full re-approval
Works offline / LAN only	No (needs ACME or paid cert)	Yes	Yes

Table 4-1. Operational impact of routine certificate operations per path.

Rule of thumb: choose the path whose *rotation story* matches your staffing. A fleet of managed Windows registers with root-ca rotation once every five to ten years is firmly Path B territory; a hosting setup where AutoSSL is already renovating certificates for you is exactly what Path A automates. Paths combine cleanly - Path B for the managed tills, Path C fallback for the owner's laptop that nobody controls.

5 Recommended Blueprint: Own CA + Wildcard + Client Bundle

Given your answers - Windows-only print clients, full admin control, Chrome/Edge, and all tenants under one root domain - Path B is the strongest fit. It needs no external CA coordination, works identically on LAN-only deployments, and turns every future certificate operation into a server-side, scriptable action. The diagram shows the complete flow.

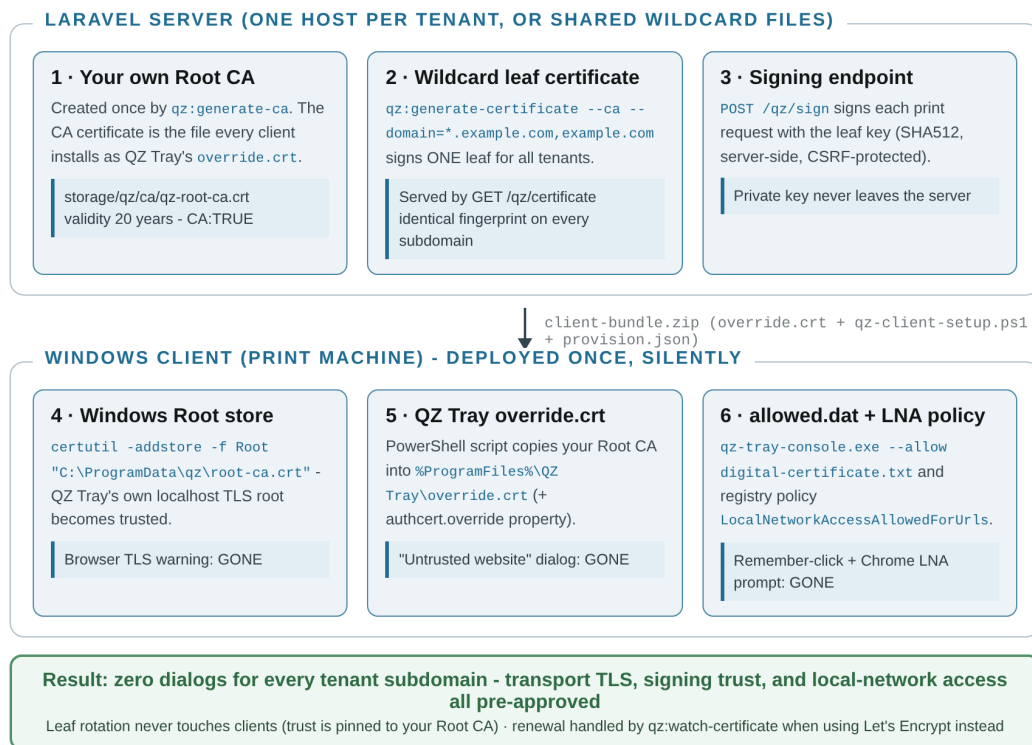


Figure 5-1. The zero-prompt architecture: one root CA, one wildcard leaf, one silent client bundle.

5.1 Server setup (once)

Four artisan commands produce every artifact the server needs. The root CA and the wildcard leaf are plain PEM files under `storage/qz`; the leaf is served automatically by the existing `/qz/certificate` endpoint on every tenant vhost, so each tenant presents the identical fingerprint. Private keys are written with 0600 permissions through atomic rename so no concurrent request can observe a half-written file.

Server-side one-time setup

```
php artisan qz:generate-ca
php artisan qz:generate-certificate --force --ca --domain=*.example.com,example.com
php artisan qz:client-bundle --zip --lna-domains=*.example.com,example.com
php artisan qz:doctor # verifies chain, SAN, fingerprint, expiry
```

5.2 Per-tenant wiring

Each tenant vhost points at the shared pair through environment variables. On a single server there is nothing to copy; on multiple servers, ship the two files with your deploy pipeline - never email or chat, and never the CA private key, which stays on the machine that created it.

Tenant vhosts share one wildcard identity

```
# every tenant vhost .env (identical values)
QZ_CERT_PATH=/home/shared/qz/digital-certificate.txt
QZ_KEY_PATH=/home/shared/qz/private-key.pem
# verification - the fingerprint must be identical everywhere:
curl -s https://tenant1.example.com/qz/status | grep fingerprint
```

```
curl -s https://tenant2.example.com/qz/status | grep fingerprint
```

Multi-tenant cheat sheet

Layer	What every tenant gets	Result on clients
Transport TLS	Nothing changes - all tenants reach the same wss://localhost:8181	One TLS trust per machine
Signing cert	The same wildcard leaf and key (shared paths or .pfx import)	One fingerprint, one trust decision
Signing trust	One override.crt per machine (or nothing, under Path A)	Zero signing dialogs
Local network	One registry policy listing the tenant domains	Zero Chrome permission prompts

Table 5-1. What tenancy changes at each layer - almost nothing.

5.3 Why the wildcard leaf matters

A wildcard SAN leaf (*.example.com) keeps every tenant visually and cryptographically uniform: the same subject, the same fingerprint, the same expiry. When QZ Tray shows the certificate in any dialog or in its Site Manager, operators see one recognisable identity instead of a different tenant name on every screen. It also means a new tenant is a routing change, not a certificate change - no issuance step exists anywhere in the onboarding path, which is exactly the property that made the pre-1.3.0 behaviour painful.

6 Package Revamp v1.4.0

The revamp is additive: every existing command, route, config key and JS contract still behaves exactly as in v1.3.0. On top of it, five new commands, two download endpoints, a browser wizard and a scheduler hook automate the trust lifecycle end to end.

6.1 New artisan commands

Command additions

Command	Purpose
qz:generate-ca	Creates the project root CA (CA:TRUE, keyCertSign, 20-year validity) at storage/qz/ca/.
qz:generate-certificate --ca --domain=*	Wildcard leaf with SAN extensions, signed by the local CA; backs up the previous pair; prints which trust path applies.
qz:override:export	Exports the file clients deploy as override.crt - the CA when the leaf chains to it, else the leaf - with deployment recipes.
qz:client-bundle	Builds the Windows bundle: override.crt, qz-client-setup.ps1, setup.bat, provision.json, README, optional .zip.
qz:watch-certificate	Syncs the signing pair from an external source (certbot/AutoSSL), validates and atomically re-imports on change; expiry guard otherwise.

Table 6-1. New commands, all additive to v1.3.0 behaviour.

6.2 New HTTP surface

Both download endpoints serve public certificate material only - no private key ever leaves the server through them - and each can be disabled by config when a fleet distributes the bundle out-of-band instead. The wizard reuses the existing security controller, so middleware overrides (for example adding auth) apply to it exactly as to the rest of /qz.

New endpoints and status posture

```
GET /qz/ca-certificate # trust root download (override.crt content)
GET /qz/client-bundle # auto-rebuilding qz-client-bundle.zip
GET /qz/setup # Client Setup Wizard (browser page)
# /qz/status now also reports:
{"trust": {
  "mode": "own-ca", # public-ca | own-ca | self-signed
  "root_ca_present": true,
  "leaf_chains_to_root_ca": true,
  "ca_certificate_url": "...", "client_bundle_url": "..."}
}
```

6.3 The Client Setup Wizard

GET /qz/setup renders a self-contained page (no CDN, published assets only) that turns the trust model into five guided steps: server posture with the SHA-1 fingerprint formatted exactly as QZ Tray displays it; the bundle download with per-step explanation of what setup.bat will silently do; a live "Test connection" probe that opens the real wss://localhost:8181 WebSocket and reports version and errors; the Path A alternative for teams who prefer zero client files; and the manual "Always Allow" fallback for unmanaged machines. A support agent can paste one URL into a chat and talk an operator to a verified-green state without any other tooling.

6.4 Configuration keys

New keys live under the qz-tray config file and are env-overridable, following the package's existing conventions. Defaults are conservative: nothing is served or scheduled unless a certificate actually exists.

```
Config additions (excerpt)
'certificate' => [
  'san_domains' => env('QZ_SAN_DOMAINS') ? explode(',', ...) : [],
  'ca' => ['cert_path' => ..., 'key_path' => ..., 'validity_days' => 7300],
  'watch' => ['source_cert' => env('QZ_WATCH_CERT'),
  'source_key' => env('QZ_WATCH_KEY'),
  'warn_days' => 30, 'schedule_time' => '03:17'],
],
'client_bundle' => ['lna_domains' => ..., 'restart_tray' => true, 'include_allow' =>
true],
'routes' => [..., 'serve_ca' => true, 'serve_bundle' => true],
```

6.5 Verification and tests

The openssl recipe was validated end to end with the OpenSSL CLI: the CA self-signs with critical CA:TRUE and keyCertSign extensions, the leaf verifies against the CA with openssl verify, the wildcard SAN survives the config round-trip, and the SHA-1 fingerprint matches the format QZ Tray prints. The test suite (CaChainAndBundleTest) encodes the same expectations against PHP openssl, and qz:doctor now reports the trust mode and zero-prompt readiness alongside its existing health checks, so a single command on any vhost answers "is this tenant serving the shared, trusted identity?"

7 Windows Client Deployment

The bundle's qz-client-setup.ps1 is the entire client story. It is idempotent - safe to run on every logon or through recurring management - and it degrades gracefully: each step detects whether it is needed and reports SKIP instead of failing. Run it once by hand on a test machine, then distribute it the same way you deploy any other startup script.

Inside qz-client-setup.ps1

Step	What the script does	Dialog it removes
0. Pre-flight	Requires elevation; locates the QZ Tray install directory.	-
1. Windows Root store	<code>certutil -addstore -f Root "C:\ProgramData\qz\root-ca.crt".</code>	Browser TLS warning on <code>wss://localhost:8181</code>
2. override.crt	Stops QZ Tray, writes the embedded CA PEM to the install folder, adds <code>authcert.override</code> to <code>qz-tray.properties</code> , restarts the tray.	"Untrusted website" signing dialog
3. allowed.dat	<code>qz-tray-console.exe --allow digital-certificate.txt</code> (belt and suspenders; skipped cleanly on older builds).	Any remaining remember-click
4. LNA policy	Sets <code>LocalNetworkAccessAllowedForUrls</code> for Chrome and Edge from the embedded domain list.	Chrome 138+ local-network permission
5. Restart	Starts QZ Tray so trust changes apply immediately.	-

Table 7-1. Bundle script steps, each guarded and idempotent.

7.1 Fleet distribution

Three proven channels, in order of preference. GPO startup script: assign the .ps1 under Computer Configuration, Policies, Windows Settings, Scripts - it runs as SYSTEM before the shell loads, which satisfies the elevation requirement. Intune/MDM: ship the .zip as a Win32 app with setup.bat as the install command. Manual per machine: right-click setup.bat, Run as administrator. In all three cases the embedded certificates travel with the script, so the client machines need no access to `/qz/ca-certificate` at deployment time.

7.2 QZ Tray provisioning sideload (2.2.4+)

For machines where you prefer the installer to deploy trust, the bundle includes a ready `provision.json`. Drop the provision folder next to the QZ Tray executable and re-run the `certgen` phase; QZ Tray performs the same steps natively during installation - useful when you already slipstream the installer for unattended rollouts.

```
provision.json - sideload into C:\Program Files\QZ Tray\provision\
[
  { "description": "Trust our signing root", "type": "ca", "phase": "certgen",
    "data": "override.crt" },
  { "description": "Whitelist site certificate", "type": "cert", "phase": "startup",
    "data": "digital-certificate.txt" },
  { "description": "Chrome: auto-allow local network", "type": "policy", "app":
    "chromium",
    "name": "LocalNetworkAccessAllowedForUrls", "format": "array",
    "data": "[\"*.example.com\", \"example.com\"]" }
]
```

8 Renewal, Rotation and Monitoring

Trust that silently expires is the final failure mode, and the one operators notice last. v1.4.0 gives each artifact a deliberate lifecycle so that nothing re-prompts and nothing expires unnoticed.

8.1 What changes when

Leaf rotation under Path B is invisible to clients: trust is pinned to the root in `override.crt`, so reissuing the leaf (new SANs, longer validity, rebranding of the subject) never touches a workstation. Path A renews every sixty to ninety days through `certbot`; the watcher re-imports the new leaf and the chain still terminates at ISRG Root X1, which QZ Tray already trusts. Only root rotation in Path B requires scheduling: redeploy the new `override.crt` bundle before switching the leaf, or accept one "Always Allow" per machine during the transition window.

Monitoring hooks

```
# routine health (wire into monitoring / cron)
php artisan qz:doctor # chain, SAN, fingerprint, expiry, DB, perms
php artisan qz:watch-certificate # exit 1 when inside the 30-day warn window
```

8.2 Backups and rollback

Every destructive operation keeps the previous artifact: `qz:generate-certificate` writes timestamped `.bak` files before replacing a pair, the watcher backs up before re-import, and `qz:certificate:import` has done so since v1.3.0. Rolling a bad rotation back is therefore a rename, not a regeneration - which matters because regenerating changes the fingerprint and would re-introduce the very prompts this architecture removes.

9 Troubleshooting Matrix and Verification Checklist

Each symptom below maps to exactly one layer. Work top to bottom: a failed transport connection makes every later layer look broken, so always start with the WebSocket probe on `/qz/setup`.

Symptom to fix

Symptom	Layer	Cause	Fix
Browser shows "not private" or the WebSocket never opens	Transport	QZ Tray root not in Windows store, or QZ Tray not running	Run bundle step 1; start QZ Tray; re-probe <code>/qz/setup</code>
"Untrusted website" dialog appears	Signing	Leaf does not chain to <code>override.crt</code> , or <code>override.crt</code> missing	Regenerate leaf with <code>--ca</code> ; re-run bundle; check <code>trust.mode</code> in <code>/qz/status</code>

Symptom	Layer	Cause	Fix
Prompt returns every session	Signing	User clicked session-only Allow; nothing was remembered	Click Always Allow once, or deploy the bundle (removes the dialog)
Prompt per tenant but not per machine	Signing	Vhosts serving different pairs	Share QZ_CERT_PATH/QZ_KEY_PATH; verify identical fingerprints
Chrome asks for local-network access	Policy	LNA policy not deployed to that browser	Bundle step 4; confirm registry value; restart browser
Signatures rejected after renewal	Signing	/qz/certificate serving a stale leaf	qz:watch-certificate --force; confirm QZ_WATCH_CERT is set
Certificate expires soon	Lifecycle	No watcher configured	Set watch paths; keep the daily schedule; qz:doctor reports days left

Table 9-1. One row per observed failure mode, mapped to its layer.

9.1 Go-live checklist

- qz:generate-ca succeeded and the CA key is 0600 and never leaves the server.
- Wildcard leaf generated with --ca; openssl verify (or qz:doctor) reports a valid chain.
- Every tenant vhost serves the identical fingerprint (curl /qz/status per tenant).
- Bundle built with the correct --lna-domains; /qz/client-bundle downloads on the tenant domain.
- Test client: setup.bat run as admin; /qz/setup probe green; test print with zero dialogs.
- qz:watch-certificate scheduled (or Path A watch paths set in .env).
- Monitoring calls qz:doctor daily; expiry warnings reach a human inbox.

Appendix A QZ Tray Quick Reference

The upstream facts referenced throughout this guide, condensed for daily use. Compatible with QZ Tray 2.2.x unless noted; the package targets the upstream 2.2.6 connector which it self-hosts.

Upstream reference

Item	Value / Command
Secure port (wss)	8181 (insecure ws: 8182)
Install shared dir	C:\ProgramData\qz
Trust a site certificate (CLI)	"C:\Program Files\QZ Tray\qz-tray-console.exe" --allow cert.pem

Item	Value / Command
Custom TLS cert (print server)	qz-tray-console.exe certgen --key privkey.pem --cert fullchain.pem
Custom TLS hostnames	qz-tray-console.exe certgen --hosts "domain1;domain2"
Override trust root (property)	authcert.override=/path/override.crt (or -DtrustedRootCert=...)
Signing algorithm	SHA512 (setSignatureAlgorithm), RSA 2048-bit key
Logs	%APPDATA%\qz ; console: qz-tray-console.exe
Provisioning sideload	C:\Program Files\QZ Tray\provision\provision.json (2.2.4+)

Table A-1. QZ Tray 2.2.x facts used by this architecture.

Appendix B New Environment Variables

All v1.4.0 behaviour is env-driven with safe defaults, following the package's existing QZ_* naming. Existing deployments upgrade without touching a single line.

Environment reference

Variable	Default	Effect
QZ_SAN_DOMAINS	empty	Comma-separated wildcard SANs for generated certificates.
QZ_CA_CERT_PATH / QZ_CA_KEY_PATH	storage/qz/ca/...	Root CA locations.
QZ_CA_VALIDITY_DAYS	7300	Root CA validity.
QZ_WATCH_CERT / QZ_WATCH_KEY	empty	External certificate source for the renewal watcher; also enables daily scheduling.
QZ_CERT_WARN_DAYS	30	Expiry warning window for qz:watch-certificate.
QZ_WATCH_TIME	03:17	Daily schedule time for the watcher.
QZ_LNA_DOMAINS	empty	Domains embedded in the Chrome/Edge LNA policy of the bundle.
QZ_BUNDLE_RESTART_TRAY	true	Whether the client script restarts QZ Tray.
QZ_BUNDLE_INCLUDE_ALLOW	true	Whether the bundle performs the allowed.dat step.
QZ_SERVE_CA_CERT / QZ_SERVE_BUNDLE	true	Enable the two download endpoints.

Table B-1. v1.4.0 additions to the existing QZ_* variable family.

B.1 Upgrading from v1.3.0

The revamp is intentionally drop-in: no migration, no JS contract change, and no behaviour change for existing endpoints. To adopt the new tooling on an installed package, update the dependency, re-publish the configuration so the new keys exist, and run the new doctor to confirm the posture detection - the certificate you already serve keeps working unchanged while you decide which zero-prompt path to adopt.

- composer update bitdreamit/laravel-qz-tray, then php artisan vendor:publish --tag=qz-config and merge the new certificate.ca / certificate.watch / client_bundle keys.
- php artisan qz:doctor - the new "Trust mode" row tells you which of the three paths you are on.
- Path B adoption: qz:generate-ca, qz:generate-certificate --force --ca --domain=... , qz:client-bundle --zip, then the bundle on one test client before fleet rollout.
- Path A adoption: qz:certificate:import with your fullchain pair, set QZ_WATCH_CERT/QZ_WATCH_KEY, confirm qz:watch-certificate exits 0.
- No template or smart-print.js change is required; the security promises and endpoints are byte-identical to v1.3.0.